



Senior AI Engineer Ürün Yol Haritası ve Teknik Mimari Analizi

Anser Enterprise AI için geliştirilen self-service doküman asistanının mevcut durumu ve altı aylık geliştirme önerisi

Temel öneri: Mevcut RAG ve LangGraph yapısı korunmalı; sistem baştan yazılmamalıdır. Üretim öncesinde kimlik doğrulama, dayanıklı ingestion işleri, kalite ölçümü ve gözlemlenebilirlik tamamlanmalıdır.

Hazırlayan

Sefa İşçi

Kapsam

Teknik değerlendirme ve proje incelemesi

Tarih

13 Temmuz 2026

İçindekiler

1	Özet
2	Mevcut sistemin değerlendirilmesi
3	Ürün geliştirme önerileri
4	Önerilen hedef mimari
5	RAG ve fine-tuning yaklaşımı
6	Monitoring ve operasyon
7	Altı aylık ürün yol haritası
8	Kod kalitesi ve riskler
9	Sonuç

Özet

İncelenen proje, teknik değerlendirmenin temel beklentilerini karşılayan çalışan bir proof of concept'tir. PDF, Word ve PowerPoint dosyaları yüklenebilmekte; belgeler iki farklı yöntemle işlenmekte; OpenAI embedding modelleri ve Qdrant üzerinden aranmakta; LangGraph akışı ile cevap ve kaynak gösterimi üretilmektedir.

Sistemin en güçlü tarafları kullanıcıya göre ayrılmış retrieval yapısı, citation doğrulaması, iki bağımsız ingestion yöntemi ve hata durumlarında güvenli cevap politikasıdır. Mevcut yapı üretim için tamamen yeniden tasarlanmamalıdır. Bunun yerine eksik olan kurumsal kimlik doğrulama, dayanıklı iş kuyruğu, gerçek değerlendirme veri seti ve uçtan uca monitoring eklenmelidir.

Kısa karar: Uygulama modüler bir bütün olarak korunmalı; ilk ayrıştırılacak servis, uzun süren ve bağımsız ölçeklenmesi gereken ingestion worker olmalıdır. LangGraph içindeki her node'un ayrı mikroservis yapılması gerekli değildir.

1. Mevcut sistemin değerlendirilmesi

Mevcut sistem React tabanlı bir web arayüzü, async FastAPI backend'i, PostgreSQL metadata veritabanı, Qdrant vector veritabanı ve OpenAI servislerinden oluşmaktadır. RAG akışı LangGraph ile yönetilmektedir.

Alan	Mevcut yaklaşım	Değerlendirme
Frontend	React, TypeScript, Vite ve Nginx	Chat, doküman yükleme, citation ve kullanım bilgileri çalışmaktadır. Kullanıcı girişi ve ayrı ingestion ekranı eksiktir.
Backend	Async FastAPI, Pydantic ve SQLAlchemy	Route, service ve repository ayrımı okunabilir ve test edilebilirdir.
Metadata	PostgreSQL	Kullanıcı, doküman, ingestion işi, sohbet ve kullanım kayıtları kullanıcıyla ilişkilidir.
Vector search	Qdrant dense ve sparse arama	Güçlü. Her sorguda kullanıcı ve tamamlanmış doküman filtresi uygulanmaktadır.
Ingestion	Semantic chunking ve Docling fixed chunking	İki yöntem ayrı tutulmuştur. Dört dosya aynı backend process'i içinde eşzamanlı işlenebilmektedir.
Agentic RAG	LangGraph retrieval ve answer alt akışları	Reranking, citation kontrolü, grounding değerlendirmesi ve tek seferlik cevap düzeltme adımı bulunmaktadır.
Monitoring	Structured log, kullanım ve maliyet kayıtları	Kısmi. OpenTelemetry trace ve gerçek kalite dashboard'u henüz yoktur.

Mevcut sistemin güçlü yönleri

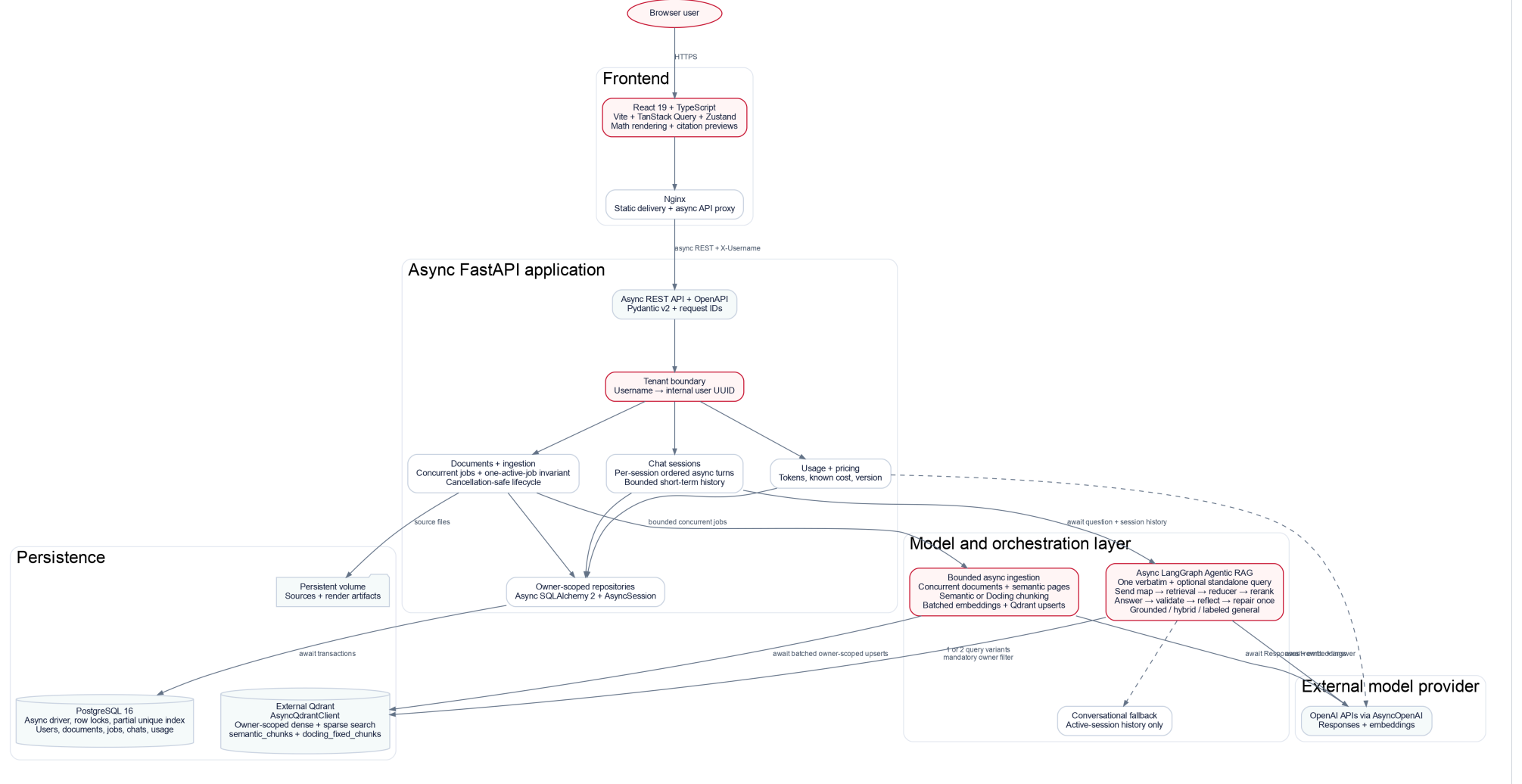
- PDF, DOCX ve PPTX desteği ile çoklu doküman yükleme bulunmaktadır.
- Semantic chunking her sayfayı bağımsız olarak değerlendirir; önceki sayfanın içeriğini yeni sayfaya taşımamaktadır.
- Docling fixed chunking ayrı bir collection ve ayrı bir processing yolu olarak korunmuştur.
- Qdrant sorgularında kullanıcı ve doküman filtreleri backend tarafından oluşturulmaktadır.
- OpenAI tabanlı reranker yalnızca yetkilendirilmiş aday chunk'ları görmektedir.
- Citation kimlikleri modelden gelen metadata'ya güvenmeden sunucu tarafında doğrulanmaktadır.
- 319 Python ve 32 frontend testi başarılıdır. Lint, production build ve Docker Compose yapı kontrolü de geçmiştir.

Üretim öncesinde tamamlanması gerekenler

- Kimlik doğrulama:** Mevcut X-Username yaklaşımı authentication değildir.
- Dayanıklı ingestion:** FastAPI BackgroundTasks process restart'ından sonra işi devam ettiremez.
- Kalite ölçümü:** Evaluation altyapısı vardır; ancak mevcut iki sentetik örnek production doğruluğunu göstermez.
- Monitoring:** OpenTelemetry, stage bazlı latency metrikleri, alert kuralları ve kalite dashboard'u eklenmelidir.
- Frontend performansı:** Production build başarılı olsa da ana JavaScript paketi büyüktür; lazy loading uygulanmalıdır.

Mevcut sistem mimarisi

THY-Branded Agentic RAG — Fully Asynchronous System Architecture



Şekil 1 — React arayüzü, async FastAPI, PostgreSQL, Qdrant, ingestion akışı, LangGraph ve OpenAI servisleri.

2. Ürün geliştirme önerileri

2.1 Kullanıcı kayıt ve giriş sayfası

Ayrı bir kullanıcı giriş sayfası eklenmesi uygundur. Ancak kurum içinde yeni bir parola sistemi yazmak yerine THY veya OREDATA'nın mevcut kimlik sağlayıcısı kullanılmalıdır. Entra ID veya başka bir OIDC sağlayıcısı üzerinden giriş yapıldıktan sonra backend token'ı doğrulamalı ve kullanıcıyı internal user_id ile eşleştirmelidir.

Bu değişiklik mevcut retrieval yapısını bozmaz. Qdrant filtreleri yine backend'in belirlediği kullanıcı kimliğiyle çalışmaya devam eder. Aynı sayfa kullanıcı rolü, workspace erişimi ve gerekli durumlarda davet sürecini de yönetebilir.

2.2 Ayrı Document Ingestion sayfası

Ingestion işlemlerinin chat ekranından ayrılması ürün açısından doğru bir adımdır. Chat ekranında hızlı yükleme seçeneği kalabilir; ayrıntılı iş yönetimi ise ayrı bir sayfada yapılmalıdır.

Bu sayfada dosya doğrulama, seçilen processing yöntemi, tahmini maliyet, sayfa ilerlemesi, oluşan chunk ve point sayısı, hata ayrıntısı, retry ve cancel işlemleri gösterilmelidir. Böylece kullanıcı başarısız bir dosyayı yeniden yüklemek yerine mevcut işi tekrar çalıştırabilir.

Mevcut dört dosyalık eşzamanlı çalışma sınırı yerel POC için makuldür. Üretimde bu sayı sabit olmamalı; worker kapasitesi, provider rate limit'i, dosya büyüklüğü ve kullanıcı kotasına göre ayarlanmalıdır. Birden fazla backend kopyası çalıştırıldığında process içi semaphore yeterli olmayacağı için ortak bir iş kuyruğu kullanılmalıdır.

2.3 Alternatif ingestion ve OCR seçenekleri

Docling mevcut sistemde korunmalıdır. Bunun yanına PaddleOCR veya PP-StructureV3 eklenebilir; ancak her dosyada bütün modelleri çalıştırmak doğru değildir. Önce belge ve sayfa özellikleri belirlenmeli, sonra en uygun yol seçilmelidir.

Belge/sayfa tipi	Önerilen yöntem	Gerekçe
Dijital, metin ağırlıklı PDF	Docling standard veya text-first parse	OCR ve vision çağırısı yapmadan daha düşük maliyet ve daha yüksek throughput sağlar.
Tarama veya düşük kaliteli görüntü	RapidOCR / PaddleOCR	Yerel OCR maliyeti düşürür; confidence değeri ölçülebilir.
Tablo ve layout ağırlıklı sayfa	Docling structure veya PP-StructureV3	Okuma sırası ve tablo yapısını daha iyi korur.
Grafik, denklem ve karmaşık görsel ilişki	Semantic vision modeli	Yalnız gerekli sayfalarda kullanılarak maliyet kontrol edilir.

PDF'in sayfa sayısı tek başına model seçme ölçütü olmamalıdır. Doğru yaklaşım; native text oranı, scan kalitesi, tablo/formül yoğunluğu, dil ve layout karmaşıklığını sayfa bazında değerlendirmektir. Büyük belgelerde sayfalar batch halinde işlenmeli ve yalnız başarısız veya değişen sayfalar yeniden çalıştırılmalıdır.

2.4 Dinamik model yönlendirme

Model routing için her soruda ayrı bir büyük model çağırısı yapmak gereksiz maliyet ve gecikme oluşturur. İlk aşamada kural tabanlı bir politika yeterlidir. Intent, istenen cevap uzunluğu, belge sayısı, retrieval sonucu, reasoning ihtiyacı ve kullanıcı bütçesi birlikte değerlendirilmelidir.

İş tipi	Model yaklaşımı
Kısa sohbet veya basit bilgi isteği	Daha hızlı ve ekonomik conversational model, düşük reasoning.
Tek belgeden kısa bilgi bulma	Ekonomik RAG cevap modeli, sınırlı çıktı uzunluğu.
Birden fazla belgenin karşılaştırılması veya uzun özet	Daha güçlü model ve orta/yüksek reasoning.
Reranking ve grounding kontrolü	Structured output üreten, düşük maliyetli ve bağımsız model ayarı.
Karmaşık semantic chunking	Sayfa karmaşıklığına göre daha güçlü modele geçiş.

Routing kararları versiyonlanmalı ve hangi nedenle hangi modelin seçildiği kullanım kayıtlarında tutulmalıdır. Yeni politika önce shadow mode'da denenmeli; gerçek değerlendirme setinde kalite kaybı göstermediği doğrulandıktan sonra otomatik hale getirilmelidir.

2.5 Mikroservis yaklaşımı

Mevcut sistem için bütün agent veya LangGraph node'larını ayrı mikroservis yapmak önerilmez. Node'lar aynı state ve hata sözleşmesini paylaşmaktadır. Bunları servisler arası çağrılara dönüştürmek ek network gecikmesi, serialization, retry ve operasyon yükü oluşturur; model çağrısı maliyetini azaltmaz.

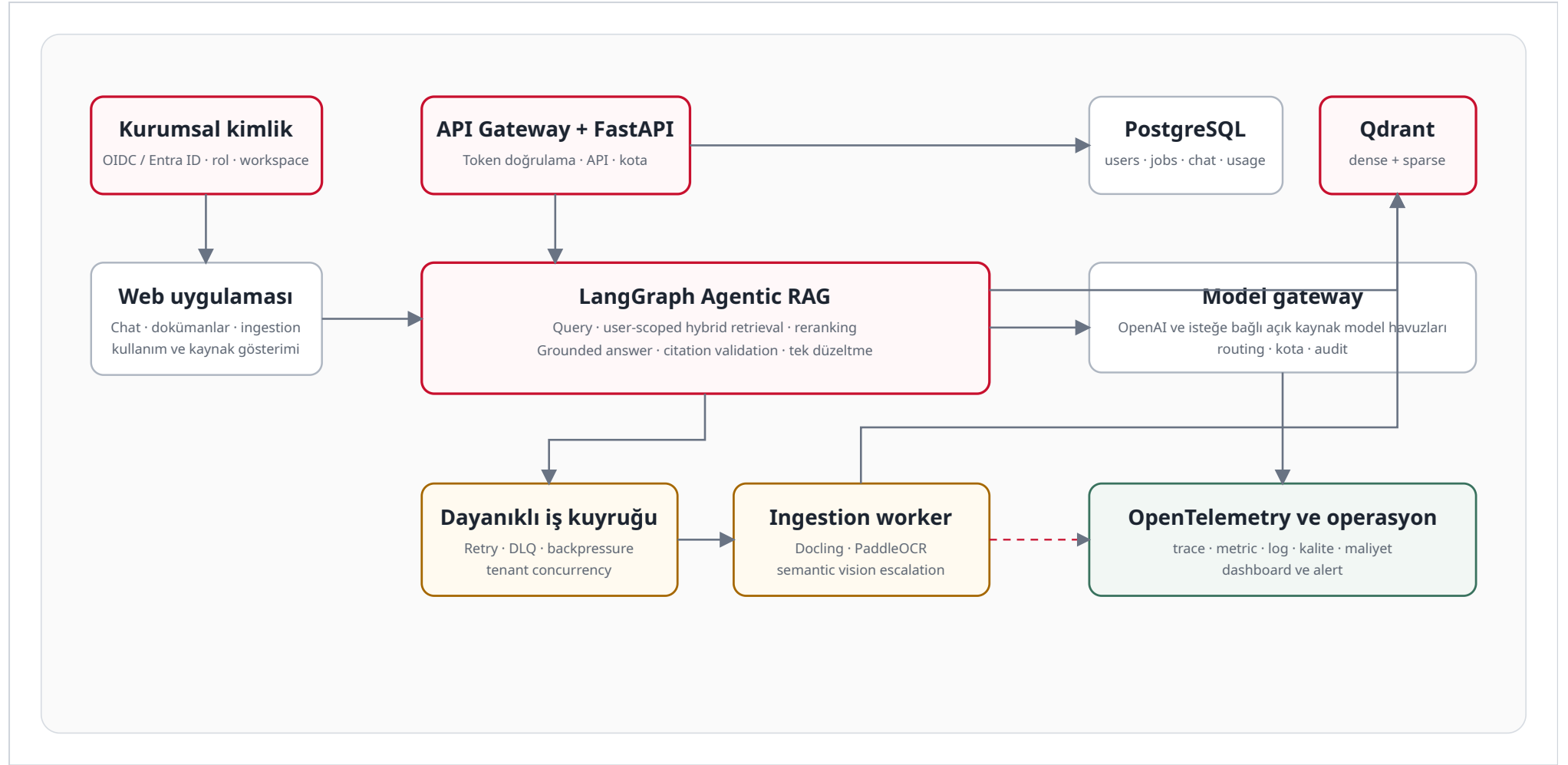
İlk ayrıştırılması gereken bölüm ingestion'dır. Bunun nedeni uzun sürmesi, chat akışından farklı kaynak tüketmesi, retry gerektirmesi ve bağımsız ölçek ihtiyacıdır. Kimlik doğrulama API gateway seviyesinde; birden fazla model sağlayıcısı kullanılırsa model yönlendirme de ortak inference gateway seviyesinde ele alınabilir.

2.6 Açık kaynak LLM

Açık kaynak model kullanılması halinde her agent için ayrı model container'ı çalıştırmak yerine ortak model havuzları tercih edilmelidir. Generation, retrieval/reranking ve document AI iş yükleri farklı havuzlarda ölçeklenebilir. vLLM gibi OpenAI-compatible bir servis kullanılması mevcut adapter yapısına geçişi kolaylaştırır.

Açık kaynak seçeneği yalnız model lisansı veya API fiyatı üzerinden değerlendirilmemelidir. GPU kullanımı, boşta bekleme maliyeti, high availability, model yükleme süresi, KV cache, güvenlik güncellemesi ve operasyon emeği toplam maliyete dahil edilmelidir. Karar, aynı THY evaluation set'i üzerinde kalite, latency, throughput ve toplam sahip olma maliyeti karşılaştırıldıktan sonra verilmelidir.

3. Önerilen hedef mimari



Şekil 2 — Mevcut RAG çekirdeği korunur. Ingestion, dayanıklı kuyruk ve bağımsız worker ile ölçeklenir. Kimlik doğrulama ve gözlemlenebilirlik tüm akışın ortak sınırlarıdır.

4. RAG ve fine-tuning yaklaşımı

Şirket dokümanlarının güncel kalması ve cevabın kaynağının gösterilmesi gerektiği için ana yöntem RAG olmalıdır. 2.000 şirket dokümanı fine-tuning ile modelin ağırlıklarına aktarılmamalı; retrieval bilgi tabanında tutulmalıdır.

Yaklaşım	Ne zaman kullanılmalı?	Bu proje için karar
RAG	Güncel, özel ve citation gerektiren bilgi	Ana yöntem. Şirket dokümanları Qdrant üzerinden erişilir.
Prompt engineering	Cevap biçimi, citation, no-answer ve tool davranışı	İlk iyileştirme adımı. Prompt versiyonlanmalı ve eval ile test edilmelidir.
Supervised fine-tuning (SFT)	Tekrarlayan format, intent veya tool-use hatası	Koşullu. Doğru davranış, tek ve güvenilir bir örnek cevapla öğretilabiliyorsa kullanılabilir.
Reinforcement fine-tuning (RFT)	Citation, groundedness, no-answer, kapsam ve tool seçiminin birlikte iyileştirilmesi	Pilot için uygun. Güvenilir bir grader kurulabilirse bu proje için SFT'den daha anlamlı olabilir.
LoRA / QLoRA	Açık kaynak modelin sınırlı donanımla uyarlanması	Açık kaynak RFT veya SFT pilotunda maliyet düşürücü seçenek olarak değerlendirilebilir.

SFT ve RFT seçimi

SFT modele iyi örnekleri taklit etmeyi öğretir. RFT ise bir cevabın ne kadar doğru olduğunu puanlayan grader üzerinden farklı çözüm yollarını karşılaştırır. Bu projede sorun yalnız cevap biçimi değil; doğru evidence kullanımı, citation doğruluğu, eksik bilgi durumunda doğru karar ve cevabın yeterli kapsamda olmasıdır. Bu nedenle ölçülebilir bir ödül fonksiyonu kurulabildiğinde RFT daha uygun bir iyileştirme yöntemi olabilir.

Ödül; citation kimliğinin gerçekten kullanılan chunk'a ait olması, iddiaların kaynaklarla desteklenmesi, sorunun kapsanması, no-answer kararının doğruluğu ve tool şemasına uyum gibi ayrı kontrollerden oluşmalıdır. İnsan değerlendirmesi yalnız öznel kalan örneklerde kullanılmalıdır. Tek bir model grader'a bağlı kalmak ve yalnız akıcı cevapları ödüllendirmek reward hacking riskini artırır.

Uygulama yaklaşımı

- 5.000 soru-cevap; eğitim, validation ve değiştirilmeden saklanan holdout evaluation setlerine ayrılmalıdır.
- Önce mevcut RAG, prompt ve retrieval kalitesi ölçülmeli; yalnız model davranışından kaynaklanan tekrar eden hatalar eğitim kapsamına alınmalıdır.
- OpenAI tarafında, organizasyon ve model erişimi uygunsa, Fine-tuning Jobs API'nin reinforcement yöntemi ile deterministic ve model tabanlı grader'lar birlikte kullanılabilir.
- Açık kaynak seçeneğinde TRL ile GRPO veya RLOO; gerektiğinde LoRA/QLoRA ve vLLM destekli bir eğitim akışı kurulabilir. Bu yol daha fazla GPU ve MLOps sorumluluğu getirir.
- SFT ve RFT aynı holdout sette citation doğruluğu, groundedness, no-answer başarısı, latency ve maliyet açısından karşılaştırılmalıdır.

Önemli ayrım: RFT de SFT gibi şirket belgelerini model ağırlıklarına taşımak için kullanılmamalıdır. Belgeler RAG katmanında kalmalı; eğitim yalnız cevaplama ve araç kullanma davranışını geliştirmelidir.

5. Monitoring ve operasyon

Canlı sistemde yalnız kullanıcı ve soru sayısını izlemek yeterli değildir. Ürün kullanımı, teknik performans, RAG kalitesi, maliyet ve güvenlik aynı request kimliği üzerinden ilişkilendirilmelidir.

Alan	İzlenecek temel metrikler
Ürün kullanımı	Aktif kullanıcı, günlük soru, yüklenen doküman, tekrar deneme, kullanıcı geri bildirimi ve en çok kullanılan belgeler.
Latency ve güvenilirlik	Uçtan uca p50/p95/p99 süre, embedding/retrieval/reranking/generation süreleri, hata ve timeout oranı, queue age ve ingestion süresi.
Retrieval kalitesi	Recall@5/10/20, MRR, reranker recall ve sonuçsuz retrieval oranı.
Cevap kalitesi	Citation geçerliliği, groundedness, cevap kapsamı, no-answer precision/recall ve insan değerlendirmesi.
Maliyet	Model ve aşama bazında token, bilinen maliyet, sayfa başına ingestion maliyeti ve kullanıcı/tenant bütçesi.
Güvenlik	Yetkisiz retrieval ve citation sayısı, authentication hatası, malware sonucu, PII olayı ve audit kaydı.

Raporlama

- Gerçek zamanlı:** Availability, hata, p95 latency, queue age, rate limit ve güvenlik alarmı.
- Günlük:** Kullanım, maliyet, başarısız ingestion işleri ve kullanıcı geri bildirimi.
- Haftalık:** Evaluation set sonuçları, retrieval ve cevap kalitesi, model/prompt karşılaştırmaları.
- Aylık:** SLO, adoption, başarılı cevap başına maliyet, güvenlik ve roadmap ilerlemesi.

OpenTelemetry ile FastAPI, HTTP client, SQLAlchemy, Qdrant ve LangGraph node'ları trace edilmelidir. Tam doküman veya kullanıcı sorusu telemetry'ye yazılmamalı; güvenli metadata, hash ve gerektiğinde kontrollü örnekleme kullanılmalıdır.

6. Altı aylık ürün yol haritası

Faz 1 — Güvenli ve işletilebilir pilot

Hafta 1–8

Kapsam	Kurumsal OIDC girişi, rol/workspace modeli, ayrı ingestion sayfası, dayanıklı iş kuyruğu, temel OpenTelemetry ve THY evaluation set v1.
Teslimatlar	Authentication/authorization, retry ve DLQ destekli ingestion worker, ilk dashboard, staging ortamı, CI ve operasyon runbook'u.
Riskler	IAM entegrasyon gecikmesi, duplicate job, evaluation verisinin yetersizliği ve PII gereksinimleri.
Bağımlılıklar	IdP erişimi, broker/object storage kararı, security onayı ve domain uzmanlarının etiketleme desteği.

Faz 2 — Kalite, maliyet ve ölçek optimizasyonu

Hafta 9–16

Kapsam	OCR/structure/vision yönlendirme, PaddleOCR deneyi, model router shadow mode, BM25 için versioned index geçişi, streaming ve feedback ekranı.
Teslimatlar	Sayfa kalite sınıflandırması, model routing policy'si, load test raporu, evaluation set v2 ve frontend bundle iyileştirmesi.
Riskler	OCR kalite farkı, router'ın ucuz modele fazla yönlendirmesi, mixed Qdrant index ve streaming cancellation karmaşıklığı.
Bağımlılıklar	Temsilî belge corpus'u, model erişimi, maliyet registry'si ve ölçülmüş kalite/latency kabul kriterleri.

Faz 3 — Enterprise pilot ve açık kaynak model değerlendirme

Hafta 17–24

Kapsam	vLLM tabanlı açık kaynak model pilotu, gerekiyorsa LoRA/QLoRA deneyi, backup/restore, disaster recovery, tenant quota ve kontrollü production rollout.
Teslimatlar	OpenAI ve açık kaynak model TCO/kalite raporu, production readiness review, restore testi, model/data card ve enterprise pilot release.
Riskler	GPU idle ve HA maliyeti, açık kaynak modelde grounding gerilemesi, GPU kapasitesi ve operasyon yükü.
Bağımlılıklar	GPU platformu, security ve data residency onayı, ölçülmüş TCO avantajı ve on-call sahipliği.

7. Kod kalitesi ve riskler

Mevcut kod okunabilir, modüler ve test edilebilir durumdadır. FastAPI tarafında route/service/repository ayrımı; model tarafında ingestion, vector store ve Agentic RAG modülleri; frontend tarafında feature tabanlı yapı bulunmaktadır.

Doğrulanmış durum

- 319 Python testi başarılıdır.
- 32 frontend testi başarılıdır.
- Frontend lint ve production build başarılıdır.
- Python compile ve Docker Compose config kontrolleri başarılıdır.
- Backend, PostgreSQL, Qdrant ve OpenAI readiness kontrolleri başarılıdır.
- GitHub repository public, MIT lisanslı ve güncel mimari görselleri içermektedir.

Başlıca riskler

Risk	Önerilen kontrol
Username üzerinden başka kullanıcı gibi davranma	OIDC token doğrulama, RBAC/ABAC ve server-side tenant mapping.
Backend restart'ında ingestion işinin kaybolması	Dayanıklı broker, idempotent worker, heartbeat, retry ve DLQ.
RAG kalite gerilemesinin fark edilmemesi	Golden evaluation set, CI quality gate, shadow ve canary release.
Qdrant schema veya sparse encoder karışması	Versioned collection, blue/green rebuild, snapshot ve rollback.
Provider maliyetinin kontrolsüz büyümesi	Global ve tenant bazlı kota, backpressure ve maliyet duyarlı routing.
OCR hatasının sessizce index'e girmesi	Sayfa kalite skoru, örnek inspection ve düşük confidence escalation.

8. Sonuç

Mevcut proje, Senior AI Engineer teknik değerlendirmesi için güçlü ve çalışan bir proof of concept'tir. RAG akışı, kullanıcıya göre filtrelenmiş Qdrant retrieval, semantic ve Docling ingestion seçenekleri, OpenAI reranker ve citation doğrulaması korunmalıdır.

Üretim yolunda öncelik yeni agent veya mikroservis sayısını artırmak değildir. İlk olarak kurumsal kimlik doğrulama, dayanıklı ingestion kuyruğu, gerçek değerlendirme seti ve OpenTelemetry tabanlı monitoring tamamlanmalıdır. Ayrı ingestion sayfası ve sayfa özelliklerine göre OCR yönlendirme ikinci aşamada eklenebilir.

Açık kaynak model veya daha geniş mikroservis mimarisi, ancak gerçek trafik üzerinde kalite, latency, ölçek ve toplam maliyet avantajı gösterildiğinde uygulanmalıdır. Bu yaklaşım mevcut yatırımı korur, teknik riski azaltır ve altı aylık süre içinde savunulabilir bir enterprise pilot oluşturur.

İlk uygulanması önerilen dört iş

1. Entra ID/OIDC ve doküman authorization sözleşmesinin hazırlanması.
2. Broker tabanlı ingestion worker ve idempotent job lifecycle tasarımı.
3. THY dokümanlarından temsili, insan etiketli evaluation set'in oluşturulması.
4. OpenTelemetry trace, metric ve kalite dashboard'unun kurulması.

Kaynaklar

1. **THY değerlendirme dokümanı:** *Senior AI Engineer Teknik Değerlendirme Çalışması.pdf*, 5 sayfa.
2. **Proje repository'si:** github.com/sefaisci/THY-Case-Study; incelenen ana dal commit'i fd2b97c7.
3. OpenAI — Choosing the right model.
4. Docling Pipelines Reference ve PaddleOCR PP-StructureV3.
5. Qdrant Hybrid Queries, LangGraph Persistence ve OpenTelemetry Python Instrumentation.
6. OpenAI Fine-tuning API ve OpenAI Graders.
7. Hugging Face TRL, vLLM OpenAI-Compatible Serving, LoRA ve QLoRA.